

Verificação de Bipartição por Busca em Largura

Prof. Me. Luis Vinicius Costa Silva

1 Problema

Considere um grafo simples e não direcionado

$$G = (V, E),$$

em que cada vértice representa um elemento e cada aresta representa um conflito entre dois elementos.

Deseja-se determinar se os vértices podem ser divididos em dois grupos de modo que nenhuma aresta conecte dois vértices pertencentes ao mesmo grupo.

Esse problema é equivalente a determinar se o grafo é *bipartido*.

Um grafo é bipartido se seu conjunto de vértices pode ser particionado em dois conjuntos disjuntos

$$V = V_0 \cup V_1, \quad V_0 \cap V_1 = \emptyset,$$

tal que toda aresta possui uma extremidade em V_0 e outra em V_1 .

2 Ideia da solução

A verificação pode ser realizada por uma busca em largura (*Breadth-First Search*, BFS) combinada com uma *2-coloração* dos vértices.

Para cada vértice v , define-se uma variável de cor:

$$cor[v] \in \{-1, 0, 1\},$$

onde:

- $cor[v] = -1$: vértice ainda não colorido;
- $cor[v] = 0$: vértice pertencente ao grupo 0;
- $cor[v] = 1$: vértice pertencente ao grupo 1.

Quando a BFS percorre uma aresta (u, v) , devem ser satisfeitas as seguintes condições:

1. se v ainda não possui cor, atribui-se a ele a cor oposta à de u ;
2. se v já possui cor e $cor[v] = cor[u]$, existe um conflito e o grafo não é bipartido;
3. se $cor[v] \neq cor[u]$, a aresta é compatível com a bipartição.

A regra de atribuição pode ser escrita como

$$cor[v] = 1 - cor[u].$$

3 Componentes desconectados

É necessário executar a BFS a partir de todos os vértices que ainda não foram coloridos.

Isso ocorre porque o grafo pode possuir mais de uma componente conexa.

O algoritmo segue a estrutura:

Algorithm 1 BFS para verificação de bipartição

Require: Grafo $G = (V, E)$

Ensure: O grafo é bipartido ou não

```
1: for all  $v \in V$  do
2:    $cor[v] \leftarrow -1$ 
3: end for
4: for all  $s \in V$  do
5:   if  $cor[s] \neq -1$  then
6:     continue
7:   end if
8:    $cor[s] \leftarrow 0$ 
9:   Inserir  $s$  na fila
10:  while a fila não estiver vazia do
11:     $u \leftarrow$  remover elemento da fila
12:    for all  $v$  adjacente a  $u$  do
13:      if  $cor[v] = -1$  then
14:         $cor[v] \leftarrow 1 - cor[u]$ 
15:        Inserir  $v$  na fila
16:      else if  $cor[v] = cor[u]$  then
17:        return DIVISÃO IMPOSSÍVEL
18:      end if
19:    end for
20:  end while
21: end for
22: return DIVISÃO POSSÍVEL
```

4 Representação por matriz de adjacência

Na primeira abordagem, o grafo é representado por uma matriz

$$A \in \{0, 1\}^{n \times n},$$

definida por

$$A_{ij} = \begin{cases} 1, & \text{se } (i, j) \in E, \\ 0, & \text{caso contrário.} \end{cases}$$

Como o grafo é não direcionado,

$$A_{ij} = A_{ji}.$$

Para verificar os vizinhos de um vértice u , percorre-se toda a linha correspondente a u :

$$A_{u0}, A_{u1}, \dots, A_{u,n-1}.$$

Um vértice v é vizinho de u quando

$$A_{uv} = 1.$$

A parte principal da BFS pode ser representada pelo seguinte pseudocódigo:

Algorithm 2 BFS utilizando matriz de adjacência

```

1:  $u \leftarrow$  remover elemento da fila
2: for  $v \leftarrow 0$  até  $n - 1$  do
3:   if  $A[u][v] = 0$  then
4:     continue
5:   end if
6:   if  $cor[v] = -1$  then
7:      $cor[v] \leftarrow 1 - cor[u]$ 
8:     Inserir  $v$  na fila
9:   else if  $cor[v] = cor[u]$  then
10:    return DIVISÃO IMPOSSÍVEL
11:   end if
12: end for

```

4.1 Implementação em C

A estrutura básica do grafo é:

```

#define MAX 100

typedef struct {
int n;
int adj[MAX][MAX];
} Grafo;

```

A inserção de uma aresta não direcionada é:

```

void adicionar_aresta(Grafo *g, int u, int v) {
g->adj[u][v] = 1;
g->adj[v][u] = 1;
}

```

A busca pelos vizinhos é realizada por:

```

for (int v = 0; v < g->n; v++) {

if (g->adj[u][v] == 0)
    continue;

```

```

if (cor[v] == -1) {
    cor[v] = 1 - cor[u];
    enfileirar(&fila, v);
}
else if (cor[v] == cor[u]) {
    return 0;
}

}

```

5 Representação por TAD de grafo com ponteiros

Uma segunda abordagem consiste em representar o grafo por listas de adjacência.

Cada vértice possui uma lista contendo seus vizinhos.

Um possível TAD é:

```

typedef struct No {
    int vertice;
    struct No *prox;
} No;

typedef struct {
    No *inicio;
} Lista;

typedef struct {
    int n;
    Lista adj[MAX];
} Grafo;

```

Nesse caso, para cada vértice u , a estrutura

$$g \rightarrow adj[u].inicio$$

aponta para o primeiro elemento da lista de seus vizinhos.

Os demais vizinhos são acessados por meio do campo

$$prox.$$

5.1 Percorrendo os vizinhos

Na matriz de adjacência, os vizinhos são encontrados percorrendo uma linha inteira da matriz.

Na lista de adjacência, percorre-se somente os elementos que realmente representam arestas:

```
No *p = g->adj[u].inicio;
```

```

while (p != NULL) {

    int v = p->vertice;

    if (cor[v] == -1) {

        cor[v] = 1 - cor[u];
        enqueue(&fila, v);

    }
    else if (cor[v] == cor[u]) {

        return 0;
    }

    p = p->prox;

}

```

A lógica da BFS permanece exatamente a mesma.
A única diferença está na maneira como os vizinhos de u são obtidos.

6 TAD da fila

Independentemente da representação utilizada para o grafo, a BFS necessita de uma fila.
Uma implementação simples pode ser definida por:

```

typedef struct {
    int dados[MAX];
    int inicio;
    int fim;
} Fila;

```

As operações fundamentais são:

- inicializar a fila;
- verificar se a fila está vazia;
- inserir um vértice;
- remover um vértice.

A fila garante que os vértices sejam processados na ordem em que são descobertos, característica fundamental da BFS.

7 Exemplo

Considere o grafo

$$0 - 1 - 2 - 3.$$

Sua matriz de adjacência é

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Inicialmente,

$$cor = [-1, -1, -1, -1].$$

A BFS começa pelo vértice 0:

$$cor[0] = 0.$$

Como 1 é vizinho de 0,

$$cor[1] = 1.$$

Em seguida, 2 recebe a cor oposta de 1:

$$cor[2] = 0.$$

Finalmente,

$$cor[3] = 1.$$

Obtém-se:

$$V_0 = \{0, 2\}, \quad V_1 = \{1, 3\}.$$

Não existe aresta entre dois vértices do mesmo grupo. Portanto,

DIVISÃO POSSÍVEL

8 Detecção de conflito

Considere agora um triângulo:

$$0 - 1 - 2 - 0.$$

A BFS pode produzir:

$$cor[0] = 0, \quad cor[1] = 1, \quad cor[2] = 1.$$

Ao analisar a aresta $(2, 0)$, verifica-se:

$$cor[2] = 1$$

e

$$cor[0] = 0.$$

Nesse caso, não há conflito.

Entretanto, a aresta $(1, 2)$ exige que os dois vértices tenham cores diferentes. Como ambos possuem cor 1, ocorre:

$$cor[1] = cor[2].$$

O algoritmo retorna imediatamente:

DIVISÃO IMPOSSÍVEL

Esse conflito corresponde à existência de um ciclo ímpar.

9 Complexidade

A complexidade depende da representação utilizada.

9.1 Matriz de adjacência

Para cada vértice processado, é necessário percorrer toda a linha da matriz para encontrar seus vizinhos.

Assim, a complexidade é

$$O(|V|^2).$$

O espaço necessário para armazenar a matriz é

$$O(|V|^2).$$

Essa representação possui como vantagem o acesso direto:

$$A_{uv}$$

pode ser consultado em tempo

$$O(1).$$

9.2 Lista de adjacência

Na representação por listas, a BFS percorre apenas as arestas existentes.

Consequentemente, a complexidade é

$$O(|V| + |E|).$$

O espaço necessário para armazenar o grafo é

$$O(|V| + |E|).$$

Essa representação é especialmente vantajosa para grafos esparsos, nos quais

$$|E| \ll |V|^2.$$

10 Comparação das representações

As duas representações implementam exatamente o mesmo algoritmo de BFS e utilizam a mesma regra de 2-coloração.

A diferença está na estrutura utilizada para encontrar os vizinhos.

Característica	Matriz	Lista de adjacência
Armazenamento	$O(V^2)$	$O(V + E)$
Consulta de uma aresta	$O(1)$	$O(\deg(u))$
Obtenção dos vizinhos	$O(V)$	$O(\deg(u))$
Complexidade da BFS	$O(V^2)$	$O(V + E)$
Implementação	Simples e direta	Ponteiros e listas encadeadas
Mais adequada para	Grafos densos	Grafos esparsos

10.1 Diferença na obtenção dos vizinhos

Do ponto de vista algorítmico, não há diferença na estratégia de resolução:

BFS + 2-coloração

é a mesma nas duas representações.

O que muda é a operação de obtenção dos vizinhos de um vértice u .

Na matriz de adjacência, percorremos toda a linha correspondente a u :

$$\text{vizinhos}(u) = \{v \mid A_{uv} = 1\}.$$

Na lista de adjacência, percorremos somente os elementos armazenados na lista de u :

$$\text{vizinhos}(u) = \text{lista de adjacência de } u.$$

Assim, em um grafo com poucos conflitos, a lista de adjacência evita percorrer posições da matriz que correspondem a pares de vértices sem arestas.

10.2 Resumo

A escolha pode ser resumida da seguinte forma:

Matriz: $O(V^2)$

Lista: $O(V + E)$

Portanto, para um grafo esperso, a lista de adjacência tende a ser mais eficiente. Para grafos pequenos ou quando se deseja uma implementação mais simples, a matriz de adjacência pode ser uma alternativa conveniente.

11 Conclusão

A verificação de bipartição pode ser realizada de maneira eficiente por BFS e 2-coloração.

O algoritmo atribui uma das duas cores a cada vértice e exige que vértices conectados por uma aresta possuam cores diferentes.

A condição fundamental é

$$(u, v) \in E \implies cor[u] \neq cor[v].$$

Se essa condição for violada, o grafo não é bipartido. Caso todas as arestas sejam compatíveis com a 2-coloração, o grafo pode ser dividido em dois grupos.

A mesma estratégia pode ser implementada utilizando uma matriz de adjacência ou um TAD baseado em listas de adjacência. A matriz oferece uma implementação direta e acesso $O(1)$ às arestas, enquanto a lista de adjacência apresenta melhor complexidade para a BFS em grafos esparsos:

$$\boxed{O(V^2) \quad (\text{matriz})}$$

e

$$\boxed{O(V + E) \quad (\text{lista de adjacência}).}$$